

END-TO-END SYSTEM VALIDATION AND
DEPLOYMENT READINESS

DIMHEX AI Doctor Technical Analysis

Comprehensive validation of the 38-subtype oncology decision-support platform, including Bayesian scoring pipeline assessment, infrastructure readiness evaluation, and a 16-week phased roadmap for production deployment transition.

System Version:

DIMHEX v4.0

Date:

July 2026

Classification:

Technical Assessment

Table of Contents

1. Executive Summary	2
1.1 Key Findings at a Glance	2
2. System Architecture Overview	2
2.1 Infrastructure Topology	3
3. Diagnostic Capacity Analysis	4
3.1 Supported Cancer Subtypes (38 Total)	4
3.2 Bayesian Scoring Engine (5D Fusion)	4
3.3 Four-Layer Decision Pipeline	5
4. End-to-End Pipeline Validation	5
4.1 Stage 1: RAG Population and Knowledge Ingestion	6
4.2 Stage 2: Scheduler and Continuous Learning Cycle	6
4.3 Stage 3: Diagnostic Query Processing	7
5. Critical Gaps and Risk Assessment	7
5.1 Critical Gaps (Blockers)	7
5.2 High-Severity Gaps	8
5.3 Medium and Low Severity Gaps	9
6. Deployment Readiness Scorecard	9
7. Deployment Roadmap	10
7.1 Phase 1: Critical Fixes (Weeks 1-4)	10
7.2 Phase 2: Integration and Testing (Weeks 5-8)	11
7.3 Phase 3: Deployment Automation (Weeks 9-12)	11
7.4 Phase 4: Operational Maturity (Weeks 13-16)	12
8. Risk Matrix and Mitigation Strategies	13
9. Conclusions and Recommendations	13

1. Executive Summary

This report presents a comprehensive end-to-end technical validation of the DIMHEX AI Doctor platform, an oncology decision-support system that integrates Retrieval-Augmented Generation (RAG), Bayesian probability engines, multi-objective optimization, and continuous learning through automated scientific literature mining. The system has evolved from supporting 8 rare cancer subtypes (v3.0) to a comprehensive coverage of 38 tumor subtypes (v4.0), representing a 4.75x expansion in diagnostic breadth. The ChromaDB vector store was successfully repopulated with 1,971 clinical records across 2 collections, and the APScheduler-based learning cycle now operates with 5 concurrent jobs including daily auto-population of 76 new synthetic cases.

The analysis validates the complete diagnostic pipeline across 4 decision layers: Bayesian therapeutic probability (Layer 1), evidence-driven feedback loop (Layer 2), Pareto multi-objective optimization (Layer 3), and clinical validation module with Oxford CEBM classification (Layer 4). The scoring engine evaluates 27 biomarkers across 5 weighted dimensions with adaptive thresholds. However, critical gaps were identified that prevent immediate production deployment, including a ChromaDB client mode mismatch between Docker configuration and application code, absent Python CI/CD pipelines, a non-functional continuous deployment workflow, and a 0% relevance rate observed during rare cancer validation testing. This report provides a phased roadmap to address these gaps and achieve production readiness within a structured 16-week timeline.

1.1 Key Findings at a Glance

Metric	Value	Status
Tumor Subtypes Supported	38 (8 rare + 30 common)	Expanded
RAG Records (ChromaDB)	1,971 (50 per subtype)	Operational
Knowledge Documents	58 scientific + 13 NCCN/ESMO protocols	Embedded
Scoring Biomarkers	27 monitored with weighted variants	Active
Decision Layers	4 (Bayesian, Evidence, Pareto, CVM)	Validated
CI/CD Pipeline	Node.js only (Python missing)	Partial
Docker Containers	7 services (fullstack)	Configured
CD Deployment Steps	Stub (echo only)	Non-functional
Rare Cancer Relevance Rate	0% in validation test	Critical Gap

Table 1: Key system metrics summary

2. System Architecture Overview

The DIMHEX AI Doctor is a dual-stack oncology platform comprising a TypeScript/Node.js frontend and a Python-based decision engine. The frontend stack leverages React 19 with tRPC 11 for type-safe API communication, Tailwind CSS 4 for styling, and Vite 6 as the build toolchain. The backend is served by Express with Helmet security headers, CORS configuration, and rate limiting set at 100 requests per 15 minutes for general endpoints and 20 requests per minute for AI-powered endpoints. Authentication is handled via JWT with role-based access control (RBAC) supporting four roles: patient, doctor, researcher, and admin. Data persistence uses Drizzle ORM with MySQL for the web stack.

The Python DIMHEX engine operates as an autonomous agent that continuously mines scientific literature from PubMed, ClinicalTrials.gov, Google Scholar, and WHO databases through a structured 7-phase research cycle executed every 4 hours. The cycle consists of: COLETAR (collect), AVALIAR (evaluate), FILTRAR (filter), SABEDORIA (wisdom synthesis), INTEGRAR (index into ChromaDB), ANALISAR (generate insights), and REPORTAR (produce executive reports). This engine populates and maintains two ChromaDB collections: `ai_doctor_tumores` for clinical case similarity search (1,971 records) and `dimhex_conhecimento` for scientific literature semantic search (71 documents). The system also implements the Senciencia v2.0 persistent memory framework with 5 memory types: episodic, semantic, procedural, projection, and deep memory.

2.1 Infrastructure Topology

The production topology is defined by `docker-compose.fullstack.yml`, which orchestrates 7 containerized services across two network segments. The frontend network connects Nginx (reverse proxy with SSL-ready configuration) to the web application container. The backend network isolates the Python DIMHEX agent, Streamlit dashboard, MySQL 8.0, Redis 7 (with append-only persistence and LRU eviction), and ChromaDB vector store. Each container has resource limits and reservations defined, with the DIMHEX agent allocated up to 2 CPU cores and 2GB RAM. Five persistent volumes ensure data durability across container restarts: `web_data`, `agente_data`, `mysql_data`, `redis_data`, and `chromadb_data`. Health checks are configured for all critical services with appropriate intervals and retry counts.

Service	Image / Build	Resources	Health Check
Nginx	nginx:1.27-alpine	0.5 CPU, 256M	30s interval
Web (React+Express)	Dockerfile (Node 22)	1.5 CPU, 1G	30s interval
DIMHEX Agent (Python)	Dockerfile.agente	2.0 CPU, 2G	60s interval
Streamlit Dashboard	Dockerfile.streamlit	0.5 CPU, 512M	None
MySQL 8.0	mysql:8.0	1.0 CPU, 1G	10s interval
Redis 7	redis:7-alpine	0.5 CPU, 512M	10s interval
ChromaDB	chromadb/chroma:latest	1.0 CPU, 1G	15s interval

Table 2: Full-stack service infrastructure

3. Diagnostic Capacity Analysis

The DIMHEX system currently supports 38 tumor subtypes with a structured Bayesian diagnostic pipeline. This section provides a detailed analysis of the diagnostic coverage, the scoring methodology, and the evidence-based validation framework that underpins each therapeutic recommendation. The system distinguishes between 8 rare cancers that were the original focus of the platform and 30 additional common cancers added during the v4.0 expansion, providing a comprehensive oncology decision-support capability that spans the majority of solid tumors and hematologic malignancies encountered in clinical practice.

3.1 Supported Cancer Subtypes (38 Total)

The 38 subtypes are organized across 15 anatomical/functional categories. Each subtype has a dedicated biomarker profile embedded in the RAG seeder, generates 50 synthetic clinical cases for ChromaDB similarity search, and has corresponding Bayesian priors for ORR (Overall Response Rate), cure probability, and toxicity probability at three treatment lines. The rare cancer cohort (8 subtypes) includes sinonasal carcinoma, bile duct cholangiocarcinoma, adenoid cystic carcinoma, tonsillar/oropharyngeal HPV-positive carcinoma, fallopian tube carcinoma, appendiceal cancer, parathyroid carcinoma, and ampullary carcinoma. The expanded common cancer cohort covers lung (NSCLC KRAS-G12C, NSCLC EGFR-mutated), breast (triple-negative, HER2-positive, HR+ luminal), prostate (hormone-sensitive, castration-resistant), pancreas (PDAC, neuroendocrine), brain (GBM, IDH-mutant astrocytoma, medulloblastoma), liver (HCC, cholangiocarcinoma), hematologic (DLBCL, follicular lymphoma, CLL, AML, APL), bone (osteosarcoma, chondrosarcoma), medullary (multiple myeloma, MDS), gastrointestinal (colorectal MSI-H, gastric HER2+, esophageal), skin (melanoma BRAF, cutaneous melanoma), genitourinary (renal cell, bladder urothelial), gynecologic (cervical HPV+), endocrine (thyroid), and pleural (mesothelioma).

3.2 Bayesian Scoring Engine (5D Fusion)

The relevance scoring engine evaluates incoming scientific findings through a 5-dimension weighted fusion model. Each dimension captures a distinct aspect of clinical applicability, and the composite score determines whether a finding is classified as Critical (score ≥ 0.75), High (≥ 0.55), Moderate (≥ 0.35), Low (≥ 0.15), or Irrelevant. The five dimensions are: Domain Pertinence (35% weight) measuring biomarker and subtype overlap with the system knowledge base; Clinical Evidence (25%) evaluating publication type, study phase, sample size, and statistical rigor; Novelty (15%) using exponential decay with a 30-day half-life to favor recent findings; Applicability (15%) assessing AI/ML integration potential and protocol reference presence; and Impact Potential (10%) quantifying survival benefit, complete response rates, and hazard ratio improvements. The engine monitors 27 biomarkers including ctDNA, CTC, TMB, PD-L1, TILs, ECOG, HPV/p16, FGFR2, IDH1, HER2, BRCA/HRD, MSI, CA-125, neuroendocrine markers, PTH, NTRK, ALK, ROS1, BRAF, EGFR, RET, MET, KRAS, TP53, and a rarity index. Each biomarker has weighted variants that capture the diverse terminology used across the scientific literature, ensuring high recall during similarity matching.

Dimension	Weight	Key Signals	Data Sources
Domain Pertinence	35%	Biomarker overlap, subtype match	27 biomarker variants
Clinical Evidence	25%	Study phase, sample size, stats	Publication metadata
Novelty	15%	Recency (30d half-life decay)	Publication date
Applicability	15%	AI/ML mentions, protocol refs	Full-text analysis
Impact Potential	10%	Survival benefit, HR, ORR	Extracted outcomes

Table 3: 5-Dimension Bayesian scoring fusion model

3.3 Four-Layer Decision Pipeline

Therapeutic recommendations are generated through a sequential 4-layer pipeline that progressively refines treatment decisions from probabilistic estimates to validated clinical actions. Layer 1 (Motor de Probabilidade Terapeutica) calculates $P(\text{ORR})$, $P(\text{cure} > 24 \text{ months PFS})$, and $P(\text{grade} \geq 3 \text{ toxicity})$ using Bayesian networks with priors extracted from NCCN/ESMO guidelines and landmark clinical trials. For example, NSCLC EGFR-mutated first-line has a prior ORR of 0.75 (95% CI: 0.70-0.80) based on the FLAURA trial with 1,078 patients. Layer 2 (Evidence-Driven Therapy) creates a feedback loop where DIMHEX findings update the Bayesian priors via Jeffreys prior formulation and DerSimonian-Laird meta-analysis. Layer 3 (Multi-Objective Optimizer) maximizes a utility function $U = 0.50 \times P_{\text{cure}} - 0.30 \times P_{\text{tox}} + 0.20 \times \text{QoL}$ subject to hard constraints ($P_{\text{tox}} \leq 0.50$, ECOG ≤ 2 for intensification, organ reserve ≥ 0.30) across 6 therapeutic actions: INTENSIFICAR, INTENSIFICAR_MODERADO, MANTER_DOSE, REDUZIR, TROCAR_LINHA, and OBSERVAR. Layer 4 (Clinical Validation Module) assigns an Evidence Index (0-100) using Oxford CEBM classification and automatically downgrades INTENSIFICAR recommendations to MANTER_DOSE when evidence is INSUFFICIENT (score < 40).

Layer	Module	Output	Clinical Foundation
1	Probabilidade Terapeutica	$P(\text{ORR})$, $P(\text{cure})$, $P(\text{tox})$	NCCN/ESMO priors + trials
2	Evidence-Driven Feedback	Updated priors	Jeffreys + DerSimonian-Laird
3	Multi-Objective Optimizer	Therapeutic action	Pareto + Goal Programming
4	Clinical Validation (CVM)	Evidence Index 0-100	Oxford CEBM + GRADE

Table 4: Four-layer therapeutic decision pipeline

4. End-to-End Pipeline Validation

This chapter traces the complete data flow from system startup through clinical case processing to therapeutic recommendation output, validating each integration point and identifying potential failure modes. The end-to-end pipeline encompasses five major stages: RAG population and knowledge ingestion, vector similarity search, Bayesian scoring and probability computation, multi-objective optimization, and clinical validation with explainability.

4.1 Stage 1: RAG Population and Knowledge Ingestion

The RAG seeder v4.0 (`rag_seeder.py`, approximately 1,400 lines) populates ChromaDB with two distinct collections. The first collection, `ai_doctor_tumores`, contains 1,971 synthetic clinical cases generated by `_gerar_vetor_biomarcador()` which produces 38 unique biomarker profiles mapped to 38 subtypes. Each case includes structured fields for subtype, biomarker vector, stage, treatment line, ECOG performance status, organ reserve metrics, and emotional state parameters. The generation process uses fixed random seeds (`random.seed(42)`, `np.random.seed(42)`) for reproducibility and creates 50 cases per subtype with realistic distributions of clinical variables. The second collection, `dimhex_conhecimento`, stores 71 documents comprising 58 scientific literature entries from PubMed-indexed journals and 13 therapeutic protocols from NCCN/ESMO guidelines. Each document has metadata for source, publication type, journal, date, authors, associated biomarkers, and relevant subtypes. The repopulation process completed in 23.3 seconds, validating the seeder performance at approximately 84 records per second.

4.2 Stage 2: Scheduler and Continuous Learning Cycle

The APScheduler-based scheduler (`scheduler.py`) operates 5 concurrent jobs that maintain the system knowledge currency. The primary DIMHEX research cycle runs every 4 hours (240 minutes), executing the complete 7-phase pipeline from literature collection through report generation. A daily auto-population job adds approximately 76 new synthetic cases (2 per subtype across 38 subtypes) to maintain RAG freshness. A memory maintenance job prunes and consolidates the Senciencia persistent memory system. A daily report generation job produces executive summaries of system findings, and a health check job runs every 10 minutes to verify ChromaDB connectivity and collection integrity. The scheduler requires a minimum of 100 cases in the tumor collection before initialization, preventing operation on an empty or degraded knowledge base. Each DIMHEX cycle queries active sources (PubMed, ClinicalTrials.gov, Google Scholar) with a 30-day lookback window, filtering results through the Bayesian relevance scorer with a minimum threshold of 0.25.

Job	Frequency	Function	Dependency
DIMHEX Research Cycle	4 hours	7-phase literature mining	>= 100 cases in RAG
Auto-Population	Daily (+76 cases)	Synthetic case generation	RAG seeder module
Memory Maintenance	Daily	Senciencia pruning	<code>dimhex_memoria.json</code>
Report Generation	Daily	Executive cycle report	DIMHEX cycle output

Health Check	10 minutes	ChromaDB heartbeat	ChromaDB connection
--------------	------------	--------------------	---------------------

Table 5: APScheduler job configuration

4.3 Stage 3: Diagnostic Query Processing

When a clinical query is submitted, the system executes a multi-stage retrieval and reasoning process. The query is first vectorized and matched against the `ai_doctor_tumores` collection using ChromaDB similarity search to retrieve the most relevant clinical precedents. Simultaneously, the `dimhex_conhecimento` collection is queried for relevant scientific literature. The retrieved cases and literature are scored by the 5-dimension Bayesian relevance scorer, which evaluates biomarker overlap, evidence quality, novelty decay, applicability potential, and impact magnitude. The scored evidence feeds into the 4-layer decision pipeline: Layer 1 computes baseline therapeutic probabilities using the pre-calibrated Bayesian priors specific to the identified tumor subtype and treatment line; Layer 2 updates these probabilities with new evidence from the DIMHEX knowledge base; Layer 3 applies the multi-objective optimizer to select the optimal therapeutic action from 6 candidates while respecting physiological constraints; and Layer 4 validates the recommendation against the Clinical Validation Module evidence index, automatically downgrading aggressive recommendations when supporting evidence is insufficient (Evidence Index < 40). The SHAP XAI explainer then generates a human-interpretable explanation of the factors driving the recommendation, providing transparency for clinical review.

5. Critical Gaps and Risk Assessment

Despite the architectural maturity and comprehensive diagnostic capabilities, the validation identified 12 significant gaps that must be addressed before production deployment. These gaps span infrastructure configuration, CI/CD completeness, scoring calibration, and inter-service communication. The following analysis categorizes each gap by severity (Critical, High, Medium, Low) and provides specific remediation guidance.

5.1 Critical Gaps (Blockers)

Gap 1: ChromaDB Client Mode Mismatch. The `docker-compose.fullstack.yml` configures the DIMHEX agent with `CHROMA_HOST=http://chromadb:8000`, indicating HTTP client mode. However, the application code in `chroma_db.py` uses `chromadb.PersistentClient(path=...)`, which operates in embedded mode with local file storage. These two modes are fundamentally incompatible: the agent container will create and use its own local ChromaDB instance, completely ignoring the shared chromadb container. This means the Streamlit dashboard and any other services connecting to the shared ChromaDB will not see the data populated by the agent, and vice versa. The health check in `docker-compose` verifies connectivity to the shared ChromaDB via `HttpClient`, but the actual application never uses it, creating a false sense of operational correctness. Remediation requires choosing a single access mode (HTTP client for multi-container shared access, or

PersistentClient with a shared volume for single-container use) and aligning both the Docker environment variables and the application code accordingly.

Gap 2: Absent Python CI Pipeline. The GitHub Actions CI workflow (ci.yml) only runs Node.js linting, type checking, Vitest tests, and Docker build verification. The Python DIMHEX engine, which constitutes the core decision intelligence of the system, has no automated testing in CI. The test files test_v3_completo.py (15 tests covering neoplasia knowledge base, biomarker scoring, Senciencia memory, and pipeline integration) and test_camadas_v2.py (layer-specific tests) exist in the repository but are never executed during continuous integration. Furthermore, pytest is not listed in requirements.txt despite being referenced in the Makefile agent-test target. This means that regressions in the scoring engine, RAG seeder, or decision pipeline can be introduced without any automated detection, posing a direct risk to diagnostic safety in a production environment.

Gap 3: Non-Functional CD Pipeline. The cd.yml workflow builds and pushes Docker images to GHCR correctly, but both the staging and production deployment steps are stubs that only echo messages. No SSH connection, Docker pull, rolling update, health verification, or smoke testing is performed. The production URL is set to the placeholder "ai-doctor.example.com". This means that while the CI/CD infrastructure appears complete from a GitHub perspective, actual deployment to any environment requires fully manual intervention. For a medical decision-support system where deployment reliability directly impacts clinical workflow availability, this gap represents a significant operational risk.

5.2 High-Severity Gaps

Gap 4: Rare Cancer Scoring Relevance Rate at 0%. Validation testing of the 8 rare cancer subtypes revealed a 0% relevance rate across all DIMHEX findings. The 5-dimension Bayesian scorer, calibrated primarily on common cancer biomarker terminology, fails to identify clinically relevant findings for rare tumor types whose biomarker profiles differ significantly from the high-prevalence cancers. This suggests that the scoring thresholds and biomarker variant lists require recalibration with rare-cancer-specific terminology and adjusted dimension weights to ensure equitable diagnostic coverage across all 38 subtypes. Without this fix, the rare cancer expansion (which represents a key differentiator of the platform) provides coverage in name only, without functional diagnostic utility.

Gap 5: No API Gateway Between Web and Agent. The Node.js web frontend and the Python DIMHEX agent run as completely independent containers with no defined communication protocol. The web application has its own Gemini-based RAG endpoint (/api/rag/oncology-query) that does not query the Python ChromaDB or invoke the DIMHEX decision pipeline. This means the sophisticated 4-layer diagnostic engine is effectively isolated from the user-facing interface. Users interacting through the web app receive Gemini-generated responses that do not benefit from the Bayesian scoring, multi-objective optimization, or clinical validation layers. An API gateway (REST or gRPC) must be implemented to bridge the web application to the DIMHEX engine, enabling end-to-end diagnostic flow from user query through the complete decision pipeline.

Gap 6: Monolithic RAG Seeder. The rag_seeder.py file is approximately 1,400 lines with 80+ hardcoded knowledge documents and 38 subtype definitions embedded inline. There is no external data pipeline, no database-driven knowledge management, and no way to update individual documents without modifying the

Python source code. This architecture makes knowledge curation, peer review, and regulatory compliance extremely difficult. For a system that claims to provide evidence-based medical recommendations, the inability to track the provenance, versioning, and approval status of individual knowledge items is a significant governance gap.

5.3 Medium and Low Severity Gaps

Gap	Severity	Description	Impact
Dockerfile Missing Files	Medium	References server_brain_analysis_endpoints.js, server_moltbook_endpoints.js (non-existent)	Build failure
Stale package.json	Low	Name "react-example" v0.0.0	Identity confusion
Unused Dependency	Low	psycopg2-binary in requirements.txt, code uses SQLite	Image bloat
Torch CPU-Only	Low	PyTorch included but only SHAP/sklearn used	2GB+ image overhead
No pytest in requirements	Medium	Tests exist but runner not installed	Manual testing only

Table 6: Medium and low severity gaps

6. Deployment Readiness Scorecard

The following scorecard evaluates the system across 12 deployment dimensions, providing a quantitative assessment of production readiness. Each dimension is rated on a scale of 0 to 5, where 5 indicates full production readiness and 0 indicates complete absence. The overall weighted score is 2.67 out of 5.0 (53.4%), indicating that the system has achieved approximately half of the requirements for production deployment but requires significant work in security hardening, testing coverage, and operational automation before it can be safely deployed in a clinical setting.

Dimension	Score	Evidence	Gap
Containerization	5/5	3 Dockerfiles, 4 compose files, 7 services	None
Health Checks	5/5	All containers monitored, appropriate intervals	None
Resource Limits	5/5	CPU/memory limits + reservations defined	None
Persistent Storage	5/5	5 named volumes for data durability	None
Security (App Level)	4/5	Helmet, CORS, rate limiting, JWT, RBAC	No HTTPS enforcement

CI Pipeline	3/5	Node.js CI complete, Docker build test	Python CI missing
CD Pipeline	1/5	GHCR push works, deploy is echo-only	No actual deployment
Inter-Service Comm	1/5	Two stacks, no API bridge defined	Web-Agent disconnected
Testing Coverage	2/5	15 Python + 7 TS + 100 stress tests	Python not in CI
Knowledge Governance	2/5	58 docs + 13 protocols embedded	All hardcoded in source
Scoring Calibration	3/5	5D fusion, 27 biomarkers, 38 subtypes	0% rare cancer relevance
Monitoring/Observability	1/5	Basic health checks only	No metrics, logging, alerting

Table 7: Deployment readiness scorecard (overall: $2.67/5.0 = 53.4\%$)

7. Deployment Roadmap

The following phased roadmap addresses all identified gaps in a prioritized sequence that maximizes clinical value delivery while systematically eliminating deployment blockers. The roadmap spans 16 weeks across 4 phases, with each phase delivering a verifiable milestone. The phrasing prioritizes safety-critical fixes first (Phase 1: Weeks 1-4), then integration and testing (Phase 2: Weeks 5-8), deployment automation (Phase 3: Weeks 9-12), and operational maturity (Phase 4: Weeks 13-16). Each phase includes specific deliverables, acceptance criteria, and risk mitigations.

7.1 Phase 1: Critical Fixes (Weeks 1-4)

Phase 1 focuses exclusively on resolving the three critical gaps that block any production deployment attempt. These are foundational issues that, if left unaddressed, would render the system non-functional or unsafe in a production environment. The ChromaDB client mode unification ensures that all services share a single source of truth for vector data. The Python CI pipeline establishment provides automated regression detection for the diagnostic engine. And the rare cancer scoring recalibration ensures that the expanded subtype coverage delivers functional diagnostic value rather than nominal coverage only.

Task	Week	Deliverable	Acceptance Criteria
Unify ChromaDB mode	1-2	HTTP client in all services	Agent + Streamlit share same data
Add Python CI to GitHub	2-3	pytest workflow in ci.yml	All 15+ tests pass on push
Fix Dockerfile references	2	Remove non-existent file copies	docker build succeeds

Recalibrate rare cancer scoring	3-4	Adjusted weights + variants	$\geq 60\%$ relevance on 8 rare types
Implement Web-Agent API	3-4	REST gateway (FastAPI/Flask)	End-to-end query flows through 4 layers
Add pytest to requirements	2	pytest + pytest-cov in requirements.txt	make agent-test succeeds

Table 8: Phase 1 task breakdown (Weeks 1-4)

7.2 Phase 2: Integration and Testing (Weeks 5-8)

With critical blockers resolved, Phase 2 focuses on building a robust testing infrastructure and externalizing the knowledge base from the monolithic seeder. The testing expansion adds integration tests that validate the complete query-to-recommendation flow across all 38 subtypes, performance benchmarks that ensure sub-second response times for similarity search, and adversarial tests that probe edge cases in the scoring engine. The knowledge externalization migrates the 71 embedded documents to a structured format (JSON/YAML) with metadata, versioning, and approval status, enabling clinical governance and regulatory compliance. This phase also implements the Web-Agent API bridge as a dedicated Python API service (using FastAPI for async performance) that exposes the DIMHEX diagnostic pipeline as RESTful endpoints callable from the Node.js web application.

Task	Week	Deliverable	Acceptance Criteria
Integration test suite	5-6	38-subtype E2E tests	All subtypes produce valid recommendations
Performance benchmarks	5-6	Latency + throughput baselines	P95 query < 2s, RAG search < 500ms
Externalize knowledge base	6-7	JSON/YAML documents with metadata	No hardcoded docs in rag_seeder.py
Knowledge versioning system	7	Git-tracked knowledge repo	Each doc has version, author, status
API bridge (FastAPI)	7-8	REST endpoints for DIMHEX pipeline	Web app calls Python engine directly
Adversarial scoring tests	8	Edge case regression suite	No NaN/Inf in probability outputs

Table 9: Phase 2 task breakdown (Weeks 5-8)

7.3 Phase 3: Deployment Automation (Weeks 9-12)

Phase 3 transforms the stub CD pipeline into a fully automated deployment system with blue-green deployment capability, automated smoke testing, and rollback mechanisms. For a medical decision-support

system, deployment automation must ensure zero-downtime transitions and immediate rollback capability to prevent clinical workflow disruption. The phase also implements comprehensive observability with Prometheus metrics, Grafana dashboards, and structured logging (JSON format with correlation IDs) to enable post-deployment monitoring and incident response. SSL/TLS termination is configured at the Nginx layer with automated certificate renewal via Let us Encrypt, and environment-based configuration management replaces hardcoded values with Kubernetes-style ConfigMaps and Secrets for the Docker Compose environment.

Task	Week	Deliverable	Acceptance Criteria
Implement real CD pipeline	9-10	SSH + Docker pull + health verify	Auto-deploy on git tag
Blue-green deployment	10	Zero-downtime rolling update	< 5s downtime during deploy
Automated smoke tests	10	Post-deploy validation suite	All smoke tests pass before traffic
Rollback mechanism	10-11	One-command rollback script	Rollback completes in < 60s
SSL/TLS with Nginx	11	Let us Encrypt auto-renewal	HTTPS enforced, HSTS enabled
Monitoring stack	11-12	Prometheus + Grafana + Loki	Dashboards for all 7 services

Table 10: Phase 3 task breakdown (Weeks 9-12)

7.4 Phase 4: Operational Maturity (Weeks 13-16)

The final phase elevates the system from technically deployable to operationally mature, addressing clinical governance, performance optimization, and disaster recovery. Clinical governance includes implementing an audit trail system that logs every diagnostic recommendation with its evidence chain, enabling retrospective analysis and regulatory compliance. The knowledge curation workflow establishes a formal process for adding, reviewing, and retiring knowledge items with clinical expert sign-off. Performance optimization targets include reducing the DIMHEX cycle time from 4 hours to 2 hours through parallel source querying and cached similarity computations. Disaster recovery procedures include automated ChromaDB backups, MySQL point-in-time recovery, and documented runbook procedures for common failure scenarios. The phase concludes with a comprehensive security audit that addresses API authentication between services, network segmentation validation, and dependency vulnerability scanning.

Task	Week	Deliverable	Acceptance Criteria
Clinical audit trail	13	Immutable recommendation logs	Every decision traceable to evidence
Knowledge curation workflow	13-14	Review/approval process for docs	Expert sign-off before indexing

Performance optimization	14-15	DIMHEX cycle < 2h, P95 < 1s	Benchmarks show 2x improvement
Disaster recovery	15	Backup + restore procedures	Full recovery in < 30 minutes
Security hardening	15-16	Inter-service auth, network audit	Zero critical vulnerabilities
Staging environment validation	16	Production-mirror staging	All tests pass in staging first

Table 11: Phase 4 task breakdown (Weeks 13-16)

8. Risk Matrix and Mitigation Strategies

The following risk matrix identifies the top operational and technical risks associated with the DIMHEX platform, along with their probability of occurrence, potential impact, and specific mitigation strategies. Risks are categorized using a standard probability-impact framework where probability ranges from Very Low (1) to Very High (5) and impact ranges from Negligible (1) to Critical (5). The risk score is calculated as the product of probability and impact, with scores above 15 requiring immediate mitigation plans and scores above 10 requiring active monitoring.

Risk	Pro b.	Imp act	Score	Mitigation
ChromaDB data loss	2	5	10	Automated daily backups, volume snapshots
Scoring calibration drift	4	4	16	Monthly recalibration, A/B testing
Web-Agent API failure	3	4	12	Circuit breaker, fallback to Gemini-only
Knowledge base staleness	3	3	9	Weekly DIMHEX cycle review, expert audit
Docker image vulnerabilities	3	3	9	Weekly Trivy scans, auto-rebuild on CVE
DIMHEX source API outage	4	2	8	Multi-source redundancy, cached results
Patient data breach	1	5	5	Encryption at rest/transit, RBAC audit

Table 12: Risk matrix with mitigation strategies

9. Conclusions and Recommendations

The DIMHEX AI Doctor platform demonstrates substantial architectural maturity with a sophisticated 4-layer Bayesian decision pipeline, comprehensive coverage of 38 tumor subtypes, and a well-designed continuous learning system. The ChromaDB vector store with 1,971 clinical records and the 5-dimension scoring engine with 27 biomarkers provide a solid foundation for oncology decision support. The infrastructure layer with 7 containerized services, resource limits, health checks, and persistent volumes demonstrates operational awareness and scalability planning.

However, the system currently operates at approximately 53% deployment readiness, with three critical blockers that prevent any production deployment attempt: the ChromaDB client mode mismatch between Docker configuration and application code, the complete absence of Python CI testing, and the non-functional CD pipeline. Additionally, the 0% relevance rate for rare cancer subtypes undermines the clinical value of the v4.0 expansion, which is positioned as a key platform differentiator. The disconnect between the web frontend and the Python DIMHEX engine means that the sophisticated diagnostic pipeline is not accessible to end users through the primary interface.

The recommended path forward follows the 4-phase, 16-week roadmap outlined in this report. Phase 1 (Weeks 1-4) should be treated as a mandatory prerequisite before any deployment consideration, as it resolves the critical gaps that affect system correctness and diagnostic safety. Phase 2 (Weeks 5-8) builds the testing and integration foundation necessary for confident production operation. Phase 3 (Weeks 9-12) automates the deployment pipeline to enable reliable, repeatable releases with zero-downtime guarantees. Phase 4 (Weeks 13-16) achieves operational maturity with clinical governance, disaster recovery, and comprehensive security hardening. Upon completion of all four phases, the system is projected to achieve a deployment readiness score of 4.5/5.0 (90%), sufficient for a controlled clinical pilot deployment with appropriate regulatory oversight and institutional review board approval.